Credit Card Fraud Detection Models using Machine Learning

Phuong Linh Le

***Abstract*—Credit Fraud Detection have always been a challenging and vital task in machine learning when coming to train a classification model. Due to the sensitivity of financial data, researchers must often rely on synthetic simulations or protected datasets to develop detection strategies. This report evaluates model performance across two distinct datasets: a generated transactional dataset provided by Chetan Mittal [1] and the widely used PCA-transformed dataset from the Machine Learning Group – ULB [2]. After a comprehensive exploratory data analysis (EDA), Logistic Regression and Random Forest models were trained. Results indicated that Random Forest outperformed Logistic Regression on both datasets, suggesting that the underlying patterns of fraud are non-linear and highly complex. Despite these findings, the study identifies several challenges—specifically regarding data imbalance and temporal drift—offering a roadmap for future improvements in real-world fraud detection systems.**

***Index Terms*—Logistic Regression, Random Forest, imbalance data**

I. Introduction

1.1. Background

In the digital economy era, the number of consumers adopting credit cards as their primary payment method is rising dramatically due to their convenience and accessibility. However, security remains a significant concern for individuals considering to use credit card. Consequently, those fraudulent transactions not only result in billions of dollars in annual financial losses but also reduce consumer trust. Developing an automated, high-precision fraud detection system is therefore no longer optional; it is a critical necessity for modern banking operations.

1.2. Challenging

Despite its importance, credit card fraud detection presents several key challenges. First, extreme class imbalance is a primary concern, as fraudulent transactions typically constitute a tiny fraction of the total data, often leading to biased models that achieve high accuracy but perform poorly in detecting actual fraud. Second, temporal patterns and data leakage create difficulties because fraud evolves over time; consequently, using a standard "stratified random split" can lead to data leakage, where future patterns are unintentionally used during training, resulting in overly optimistic performance estimates. Finally, capturing customer and merchant behaviors is essential, requiring sophisticated feature engineering and model selection to effectively distinguish legitimate purchases from fraudulent ones based on underlying activity patterns.

1.3. Objectives

This report aims to evaluate and compare the efficacy of variety classifiers across two distinct data architectures:

- Transactional Data: reserves original transactional patterns, requiring extensive "Feature Engineering" pipeline to extract behavioral patterns such as transaction frequency, geographical distance, and spending ratios.
- PCA-Transformed Data: numerical variables resulting from Principal Component Analysis (PCA), which representing a scenario where features are anonymized and numerically encoded

To address these challenges, I applied a structured pipeline that includes data exploration, feature processing, model training and performance evaluation using appropriate metrics for imbalanced classification.

The primary objective is to optimize the F1-Score and PR-AUC, ensuring the model achieves a robust balance in the trade-off between maximizing fraud detection (Recall) and minimizing false alarms (Precision).

II. Related Work

Recent literature applies various approaches to imbalanced financial datasets, each with specific strengths and weaknesses. Dornadula and Geetha (2019) [3] demonstrated that while Random Forest is robust, its performance improves significantly when coupled with SMOTE. However, Taha and Malebary (2020) [4] proposed a CNN-based hybrid deep learning approach, arguing that traditional methods like Logistic Regression fail to capture complex, non-linear boundaries in highly imbalanced fraud datasets. Feature engineering also remains a point of debate: Benchaji et al. (2021) [5] noted that while PCA aids in noise reduction and privacy, it may discard "minority variance" containing critical fraudulent signals. Conversely, Whitrow et al. (2019) [6] proved that temporal aggregate features, such as spending frequency over sliding windows, are more predictive than individual transaction attributes. Regarding evaluation, Dal Pozzolo et al. (2018) [7] highlighted that stratified random splits yield over-optimistic results, advocating for Chronological Splitting to simulate real-world production. Furthermore, Bhattacharyya et al. (2011) [8] argued that the Area Under the Precision-Recall Curve (PR-AUC) is a more reliable metric than ROC-AUC, which can be misleadingly high due to the large number of True Negatives.

Despite extensive research, a gap remains in comparing PCA-anonymized data against raw transactional data under identical temporal constraints. This research addresses this

void by evaluating how feature transparency versus abstraction impacts PR-AUC and F1-Score in a production-ready, leakage-free pipeline.

III. Methodology

The framework consists of 5 steps of CRISP-DM Framework was adopted in this project

3.1. Business Understanding

Credit card fraud is a critical challenge for banking institutions, occurring at a massive scale every second. The primary objective is to leverage historical transaction data to develop a proactive defense mechanism. This system aims to:

- Reduce financial loss: Identify and block fraudulent activities before the transaction is finalized.
- Protect Customers: Provide real-time alerts to cardholders, enhancing trust and security.
- User Experience: Ensure that legitimate transactions are not mistakenly blocked (Minimize the false alarm)

To meet the business objectives, the Machine Learning model must satisfy three technical criteria:

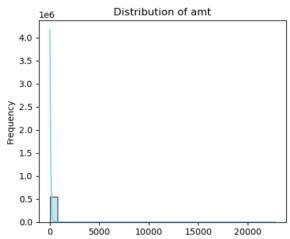
- High Recall: Accurately detecting as many fraud cases as possible (minimizing the rate of undetected fraud).
- High Precision: Maintaining a low False Positive rate to ensure that the user experience is not compromised by unnecessary transaction declines.
- Low Latency: The model must process and return a prediction in milliseconds.

3.2. Data Understanding

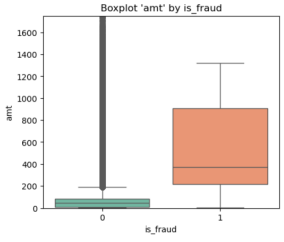
3.2.1. Dataset 1: The dataset consists of 22 features, including 10 numerical variables, 12 categorical variables, and a binary target variable, is_fraud, where a value of 1 indicates a fraudulent transaction. No missing values or duplicate records were identified, allowing the analysis to proceed directly to exploratory data analysis.

The features can be broadly categorized into transactional attributes (e.g., amt, trans_date_trans_time), customer or contextual information (e.g., dob, category, merchant), and geospatial variables such as latitude and longitude.

The amt (transaction amount) feature exhibits a strong right-skewed distribution, with values ranging from 1.0 to 22,768.1. This indicates the presence of extreme values (outliers), suggesting that data transformation or scaling techniques may be required to mitigate their impact on model performance.



(a) Histogram of transaction amount



(b) Boxplot of transaction amount by class

Fig. 1: Distribution and comparison of transaction amounts

Furthermore, a boxplot comparing transaction amounts between legitimate and fraudulent transactions suggests that amt is a potentially important feature for classification, as the distributions differ noticeably between the two classes.

A significant class imbalance is observed between legitimate and fraudulent transactions, with fraudulent cases accounting for only 0.34%.This imbalance implies that accuracy alone is not an appropriate evaluation metric, and alternative metrics such as precision, recall, or F1-score should be considered.

Additionally, the dataset exhibits a temporal nature, meaning that transactions are sequential and potentially dependent on historical patterns. This is particularly relevant for fraud detection, where past behavior can help identify anomalous activity. Therefore, appropriate data-splitting strategies (e.g., time-based splitting) should be considered to avoid data leakage during model training and validation.

3.2.2. Dataset 2: This dataset contains 28 features derived from Principal Component Analysis (PCA), meaning all features are numerical. The transactions were recorded over a two-day period. The Time feature represents the number of seconds elapsed since the first transaction, while the Amount feature retains the original transaction value. The target variable, Class, indicates whether a transaction is fraudulent.

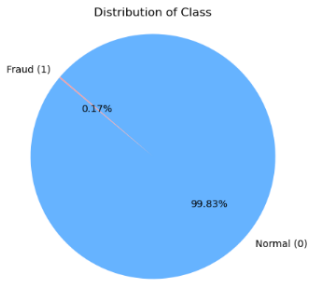


Fig. 2: 'Class' feature distribution pie chart

Similar to Dataset 1, this dataset exhibits a severe class imbalance. Although there are no missing values, 1,081 duplicate records were identified and removed during preprocessing.

A correlation matrix analysis indicates no strong multicollinearity among the PCA-derived features or the Amount variable, which is expected due to the orthogonal nature of PCA components. The use of PCA limits feature interpretability but reduces dimensionality and noise.

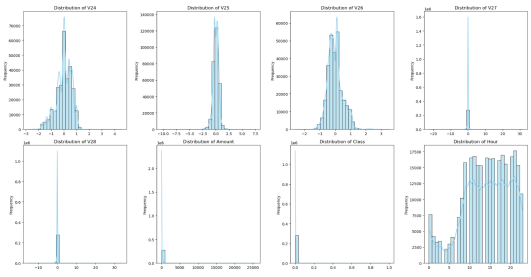


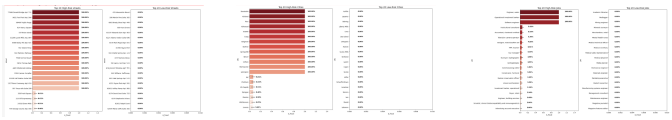
Fig. 3: Dataset 2: Features histogram chart

The histogram plots reveal that all PCA-derived features, excluding 'Time' and 'Amount', have been centralized to 0. Consistent with Dataset 1, the 'Amount' variable exhibits a strong right-skewed distribution. The majority of transactions involve small values, which explains the high frequency (tall bars) near zero. Furthermore, the histograms indicate that, in addition to 'Amount', several 'V' features display significant skewness and are heavily influenced by outliers. While the primary distribution of these features is concentrated in the center, the presence of heavy tails (stretching far to the sides) suggests a high degree of kurtosis. Therefore, appropriate preprocessing methods should be adopted before applying a model to them.

3.3. Data Preparation

3.3.1. Dataset 1: In the data preparation stage, temporal values were created from the transaction timestamp, and an age feature was calculated instead of using date of birth to help the model find relationships with the current time

High cardinality in categorical features, such as street, city, and job, was identified as a significant risk for dimensionality explosion and overfitting during one-hot encoding. These features were excluded, particularly where certain categories exhibited biased 100% fraud rates that would skew model generalization. Additionally, the state column was removed to eliminate redundancy, as merchant locations were already precisely defined by longitude and latitude. Analysis of the



(a) By Street **(b)** By State **(c)** By Job

Fig. 4: Analysis of Fraud Rates across different categories

stacked column charts (Fig.6) showed no difference in gender distribution between transaction types, so gender was dropped as a non-informative feature. To handle the complex-

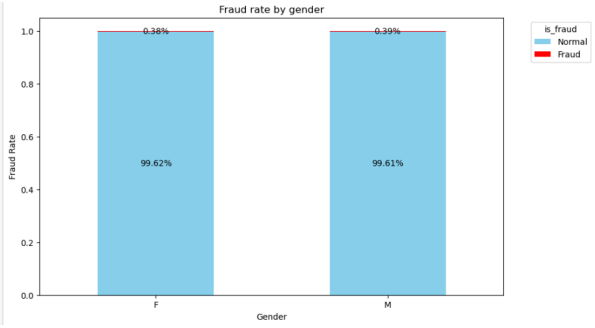


Fig. 5: 'Gender' feature distribution column chart by 'is_fraud'

ity of the merchant feature,frequency-based metrics including merch_exp_count and merch_velocity_24h were extracted to capture transaction frequency.

Finally, the numeric features, including both the original dataset variables and the newly derived ones, which can be

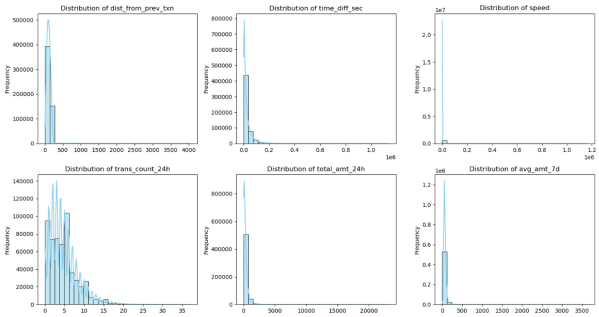


Fig. 6: Histogram for a subset of features dataset 1

divided into three groups: historical amount features (such as amt_vs_historical_ratio) to compare current and past spending; and spending habits (such as trans_count_24h and ratio_vs_7d) to detect abnormal behavior. After removing redundant variables, I addressed distribution skewness, which can unfairly bias the model. Using a skew index and histograms, I identified features with skewness over 75% - mostly amount-related—and applied a log transformation to compress their range. Next, I applied specific scaling and encoding based on feature characteristics. Robust Scaling was used for skewed features like 'amt' and 'ratio_vs_30' to handle outliers as long as the long tail via the median and IQR, while Standard Scaling was applied to the rest. Time-related features were processed using a sin-cos transformation to capture their cyclical nature. Finally, I binned the age feature into four categories to help the model identify broader group patterns rather than individual noise.

After all of above, a feature selection Random Forest method along with 'balance' class_weight support feature was run to choose the upmost features for model which can improved the performance of model dramatically comparing to over 30 features.

	feature	importance
9	amt	0.197408
10	total_amt_24h	0.146303
11	avg_amt_7d	0.113181
12	amt_vs_historical_ratio	0.106364
0	hour	0.088761
13	ratio_vs_30d	0.083398
6	category	0.045826
14	ratio_vs_7d	0.045480
15	avg_amt_30d	0.042987
16	amt_perc_of_24h	0.022652
5	location	0.018511
17	time_diff_sec	0.015912
18	speed	0.012460
19	trans_count_24h	0.010491
1	day	0.006894
3	minute	0.006863
20	city_pop	0.005715
21	merch_exp_count	0.005588
4	day_of_week	0.005006
22	dist_from_prev_txn	0.004614
23	distance_km	0.004288
2	month	0.003656
24	merch_velocity_24h	0.002602
7	age_group	0.002255
8	gender	0.001382
25	is_first_txn	0.000962
26	is_weekend	0.000441
27	is_invalid_latlong	0.000000

Fig. 7: Random Forest Feature Selection on dataset 1

From the table, I chose out 14 group of features which

contribute over 1% to fraud classification.

3.3.2. Dataset 2: First, the hour was extracted from the time column to capture repeatable temporal patterns, followed by sine-cosine encoding to preserve these cyclical characteristics. After analyzing outlier rates and skewness indices, we identified features sensitive to extreme values and applied Robust Scaling. Specially, Amount feature has

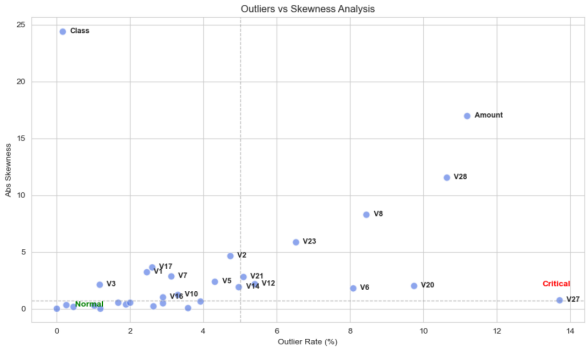


Fig. 8: Scatter plot skewness vs outlier rate dataset 2 features

a extreme right skewness along with wide range, therefore a log transformation should be used with the robust scaling to compress the difference. And finally, the same as dataset 1, a feature importance measuring method using random forest was conducted with the result shows as below.

	feature	importance
8	V14	0.188003
18	V4	0.102232
11	V12	0.097418
5	V10	0.094253
14	V17	0.086428
20	V11	0.069860
7	V16	0.066686
2	V7	0.035594
10	V3	0.035101
15	V2	0.025665
13	V1	0.020418
23	V18	0.019467
19	V9	0.017769
0	Amount	0.016017
9	V20	0.010966
3	V21	0.010612
24	V19	0.010389
12	V5	0.010240
21	V13	0.008358
6	V27	0.008172
17	V8	0.007971
1	V28	0.007838
4	V6	0.007401
22	V15	0.007337
16	V23	0.007310
25	V22	0.007266
28	V26	0.006361
27	V25	0.005499
29	Hour	0.004944
26	V24	0.004425

Fig. 9: Dataset 2: Feature Importance table (RF)

From this feature importance list (Fig.10), we can tell that the importance is split to all PCA features, which is because of the way PCA work - extract the helpful information from the original features, compress it into smaller number of uncorrelated components, which means every PCA variable carry at least some helpful information to help the model. However, it is recommended to conduct a test on compact model after drop some less importance feature

3.4. Modeling

In this stage, two distinct modeling approaches were employed to capture the relationships between features and the target variable. For the linear approach, Logistic Regression (LR) was selected due to its efficiency on large datasets and interpretability. For the non-linear approach, Random Forest (RF), a tree-based ensemble method, was utilized to discover complex patterns and interactions. To address the severe class imbalance, a hybrid resampling strategy was implemented.

- Random Undersampling was applied to reduce the majority class until the minority class represented 10% of the majority class's volume ($n_{minor} = 0.1 \times n_{major}$). This step reduces computational cost and prevents potential memory issues while retaining a significant portion of majority class information.
- SMOTE (Synthetic Minority Over-sampling Technique) was used to further balance the classes by generating synthetic samples for the minority class.

Furthermore, to account for the temporal nature of financial transactions, TimeSeriesSplit was used for splitting the test set. This ensures that the model is always trained on past data and validated on future data, preventing data leakage and ensuring a realistic evaluation.

To ensure the model's stability and generalizability, K-Fold Cross-Validation (specifically implemented as TimeSeriesSplit) was employed. This approach evaluates the model's average performance across multiple unseen data folds, confirming that the detector remains reliable across different time periods and scenarios. By simulating a rolling forecast origin, this validation technique minimizes the risk of over-fitting and provides a more robust estimate of how the model will perform in a real-world production environment.

3.5. Evaluation

Relying solely on Accuracy in credit card fraud detection can be highly misleading. In an imbalanced dataset, a model could achieve 99.9% accuracy by simply predicting all transactions as "safe," thereby failing the primary objective of detecting fraud (True Positives). Consequently, the following metrics were prioritized:

- **F1-Score**: The harmonic mean of Precision and Recall, balancing the trade-off between reducing false alarms and capturing as many fraudulent cases as possible.

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **Precision-Recall AUC (PR-AUC)**: Unlike the ROC-AUC, the PR-AUC is more robust for imbalanced classification as it focuses on the performance of the minority class. A higher PR-AUC indicates that the model maintains high precision across various recall thresholds, which is crucial for flexible business decision-making.

IV. Implementation

The implementation was carried out using the Scikit-learn and Imbalanced-learn libraries in Python. A robust pipeline was constructed to ensure a seamless flow from raw data to model prediction, preventing any potential data leakage during the resampling and scaling phases.

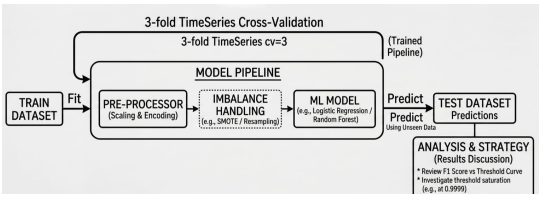


Fig. 10: Project pipeline

The training process focuses on handling severe class imbalance and optimize model performance through a three-stage process. This framework is applied consistently to both LR and RF models.

Imbalance Handling: To address the disproportionate class distribution, the pipeline is engineered to evaluate and integrate different resampling and cost-sensitive techniques:

- Cost - Sensitive Learning: Utilizing the class_weight = 'balanced' or 'balanced_subsample' parameter to adjust the model's penalty for misclassifying the minority class.
- Resampling Methods: Integration of Hybrid Sampling (Random Undersampling + SMOTE) and SMOTE Oversampling at various ratios (e.g., $\{0.2, 0.3, 0.5\}$).

Hyperparameter Tuning: the system refines the model structure:

- Search Strategy: A RandomizedSearch CV framework is implemented to explore the hyperparameter space. For Random Forest, this includes tuning max_depth, min_samples_leaf.
- Efficiency: Randomized search is selected over Grid Search to maintain a balance between exhaustive exploration and computational efficiency, given the large-scale nature of the dataset.

Threshold Tuning: Recognizing that the standard 0.5 decision threshold is often ineffective for imbalanced datasets, the final stage of the implementation involves a dynamic threshold adjustment. By analyzing the Precision-Recall Curve on these validation sets, the implementation identifies the specific threshold that maximizes the F1-Score, which is then applied to the final test set.

Crucially, to ensure a realistic real-time scenario, the entire training and tuning process is conducted exclusively on training set using a 3-fold TimeSeriesSplit cross-validation scheme.

V. Simulation

5.1. Dataset 1

5.1.1. Logistic Regression Model: To ensure model stability, Variance Inflation Factor (VIF) analysis and correlation matrix was conducted to identify and remove features exhibiting severe multicollinearity (VIF 10). This process optimized the feature set by reducing redundancy, ensuring that the regression coefficients accurately reflect the predictive impact of each variable on fraud detection

Features derived from the 'amount' variable exhibit strong correlations, while a near-absolute relationship exists between the 'long' and 'lat' features. Nevertheless, the feature importance ranking indicates that most amount-based features are critical predictors. Consequently, only 'ratio_vs_30d' was removed; this decision effectively reduced multicollinearity while preserving the long-term historical information shared by 'ratio_vs_30d' and 'amt_vs_historical_ratio'. Additionally, the 'long' and 'lat' features were excluded based on the rationale that user locations are inherently dynamic rather than static. After eliminating these features, the VIF scores fell within an acceptable range:

	Feature	VIF
3	amt_vs_historical_ratio	9.324611
1	amt	6.117060
6	ratio_vs_7d	5.415677
4	avg_amt_7d	1.993555
8	avg_amt_30d	1.843943
12	trans_count_24h	1.786880
2	total_amt_24h	1.728531
11	time_diff_sec	1.280686
7	amt_perc_of_24h	1.081371
5	hour	1.009102
13	speed	1.001542
9	merch_lat	1.000625
10	merch_long	1.000344

Fig. 11: VIF Score on dataset 1

Subsequently, a comprehensive pipeline - incorporating Random Undersampling, SMOTE, and Class Weights - was applied to the selected features to evaluate the model's response to different balancing techniques. The results across various SMOTE ratios are summarized below:

```
Start running Logistic Classification on 14 features...
--------------------------------------------------
SMOTE Ratio 0.2:
  > Mean F1 Score: 0.0866
  > Mean PR-AUC:   0.5556
------------------------------
SMOTE Ratio 0.3:
  > Mean F1 Score: 0.0860
  > Mean PR-AUC:   0.5551
------------------------------
SMOTE Ratio 0.5:
  > Mean F1 Score: 0.0875
  > Mean PR-AUC:   0.5528
------------------------------
```

Fig. 12: Logistic model 1 result on validation sets (1)

The results were suboptimal, with F1-scores remaining consistently low (< 0.1) across all ratios. To investigate this further, several LR pipelines were conducted using different feature sets—specifically comparing the full feature set against the Random Forest-selected features—combined with dynamic imbalance handling methods.

Despite testing four different model configurations, the F1-score fluctuated around 0.08 without significant improvement. This suggests that the relationship between fraud probability and the predictors is likely non-linear, which prevents a linear estimator like LR from capturing complex patterns. Ultimately, the configuration utilizing 14 features and a hybrid sampling approach (SMOTE ratio 0.2 + Random Undersampling + Class Weights) was selected for the threshold tuning stage, as it demonstrated a stable mean PR-AUC and a competitive F1-score relative to the other trials.

By evaluating the model across three validation sets, the optimal threshold for maximizing the F1-score was identified

at 0.9815 - a peak F1-score of 0.5696. Finally, the LR model, configured with the optimized threshold, was applied to the hold-out test set. This evaluation serves to determine the model's peak performance and its practical effectiveness in classifying fraudulent transactions.

5.1.2. Random Forest Model: First of all, two RF model using the same parameters for two difference class_weight type was conducted:

Type	Mean F1 Score	Mean PR-AUC
'balanced'	0.5957	0.7858
'balanced_subsample'	0.5251	0.7745

TABLE I: Class weight performance on validation sets (1)

There is no significant performance between them, which indicates that the global class proportions are sufficiently representative, making the dynamic re-calculation of weights unnecessary. Consequently, the standard 'balanced' approach was adopted for its computational efficiency and stability. Therefor, a random search was used to random training and validate 5 set of parameter on 3 fold of dataset. The min_samples_leaf was set to 4 and 10 to prevent the model from capturing noise or individual outliers. The max_depth parameter controls the complexity of the decision boundaries. By testing values from 10 to 30, we aimed to find the threshold where the model captures complex fraud patterns without memorizing the training noise.

	max depth	min samples leaf	mean test PR AUC	mean test F1	F1 on train set
1	30	10	0.8537	0.7948	0.9074
4	20	10	0.8542	0.7917	0.8904
3	None	4	0.8696	0.7906	-
0	20	4	0.8615	0.7890	-
2	10	4	0.7833	0.5880	-

TABLE II: Result optimization Random Forest Model (1)

It is obvious that when the depth beyond 20, F1 on validation test very promissing but led to a significant gap between training and testing performance, suggesting the onset of overfitting. Therefore, in the next random search, the distribution of depth was limited between 10 and 15.

Reducing the depth force them to create a general model that can endeavor any unseen transaction, the model with depth of 15 and leaf of 10 remains a impressive F1 score (0.7617) while pull the gap (0.058) between validation set and training set to an acceptable number, therefor this set of parameter was chose as a final model using to tuning threshold

Same to the RF model using hybrid sampling, a random search along with cross-validation led to the result as below

	max Depth	min Samples Leaf	mean Test PR AUC	mean Test F1	F1 on train set
3	20	4	0.873479	0.769252	0.8537
2	20	10	0.865946	0.686974	
0	15	10	0.860435	0.668712	
1	15	10	0.860435	0.668712	
4	12	4	0.846911	0.641566	

TABLE III: Randomized Search Results for RF Optimization + hybrid sampling

After deciding the best set of parameter for both models, a 3-fold Time-Series Cross-Validation (TSCV) was employed to identify the optimal decision threshold. The prediction probabilities from all validation folds were aggregated to compute a averaged Precision-Recall Curve using precision_recall_curve function from sklearn.metrics. The optimal threshold for the RF model with cost-weighted adjustments was determined to be 0.5690 by maximizing the F1-score across validation folds, rather than relying on the default 0.5 cutoff. This adjustment yielded a peak F1-score of 0.7855

In comparison, the RF model employing a hybrid sampling approach (SMOTE ratio 0.1, Random Undersampling 0.01, and Cost-Weighting) performed best at a threshold of 0.6684. This second configuration achieved a superior average F1-score of 0.8167 across the 3-fold validation process.

Based on these results, both RF models yielded impressive performance. Consequently, in the final stage, both models were deployed to predict the test set to evaluate their generalization capabilities on real-world, unseen transaction data.

5.2. Dataset 2

5.2.1. Logistic Regression Model: Following the approach taken for the first dataset, multiple imbalance-handling methods were applied alongside, yielding the following results:

```
Start running Logistic Classification on 30 features...
--------------------------------------------------
SMOTE Ratio 0.2:
  > Mean F1 Score: 0.0536
  > Mean PR-AUC:   0.6193
------------------------------
SMOTE Ratio 0.3:
  > Mean F1 Score: 0.0599
  > Mean PR-AUC:   0.6063
------------------------------
SMOTE Ratio 0.5:
  > Mean F1 Score: 0.0610
  > Mean PR-AUC:   0.6305
------------------------------
```

(a) Hybrid sampling

```
Start running Logistic Classification without hybrid sampling on 30 features...
--------------------------------------------------
  > Mean F1 Score: 0.0718
  > Mean PR-AUC:   0.7381
----------------------------
```

(b) Class weight adjustment

Fig. 13: LR perfomance on validation sets (2)

Once again, the LR model yielded disappointing results, with the maximum F1-Score reaching only 0.07.

```
Start running Logistic Classification on 18 features...
--------------------------------------------------
SMOTE Ratio 0.2:
  > Mean F1 Score: 0.0823
  > Mean PR-AUC:   0.6758
------------------------------
SMOTE Ratio 0.3:
  > Mean F1 Score: 0.0858
  > Mean PR-AUC:   0.6637
------------------------------
SMOTE Ratio 0.5:
  > Mean F1 Score: 0.0834
  > Mean PR-AUC:   0.6738
------------------------------
```

(a) Hybrid sampling

```
Start running Logistic Classification without hybrid sampling on 18 features.
--------------------------------------------------
  > Mean F1 Score: 0.1031
  > Mean PR-AUC:   0.7060
----------------------------
```

(b) Class weight adjustment

Fig. 14: LR perfomance on validation sets (2)

Subsequently, LR was applied to a subset of features. These were identified using the Random Forest Feature Selection method, specifically retaining only those with an importance score greater than 0.01.

Results (Fig.14) indicate that the reduced-feature model performed better (peak F1-Score of 0.1). However, since the hybrid sampling approach did not yield a significant improvement over simple weighting, the cost-weighted LR model was selected for the threshold tuning stage. Following a 3-fold time-series cross-validation, the optimal threshold was identified as 0.9987, yielding the highest F1-Score of 0.7346.

5.2.2. Random Forest Model: Attention was then turned to hyperparameter tuning, a vital step in controlling how deeply the RF model learns from the training set. To control model depth and prevent overfitting, two iterations of Random Search were conducted. While initial configurations led to significant overfitting (training F1-Scores 0.92), a second iteration with narrower hyperparameter ranges (Fig. 15) was implemented to ensure better generalization on the test set.

	param_rf__max_depth	param_rf__min_samples_leaf	mean_test_F1	\
3	15	10	0.811236	
4	10	10	0.799570	
0	12	50	0.779417	
2	15	100	0.714274	
1	12	100	0.707570	

	mean_train_F1	mean_test_PR_AUC
3	0.905196	0.770949
4	0.901693	0.773536
0	0.884003	0.767376
2	0.848629	0.760812
1	0.845192	0.759545

Fig. 15: Random search RF Optimzation (2)

After evaluating the candidates on the full training set, the configuration with max_depth: 10 and min_samples_leaf: 10 was selected. This model demonstrated the most acceptable balance, yielding a training F1-Score of 0.9040 compared to 0.7995 on the validation set, it maintained a smaller performance gap (0.10) compared to other versions, which all showed gaps wider than 0.11.

VI. Results

6.1. Dataset 1

---RESULT ON TEST SET---

	precision	recall	f1-score	support
0	1.00	0.94	0.97	110960
1	0.02	0.81	0.04	184
accuracy			0.94	111144
macro avg	0.51	0.88	0.51	111144
weighted avg	1.00	0.94	0.97	111144

(a) Threshold = 0.5

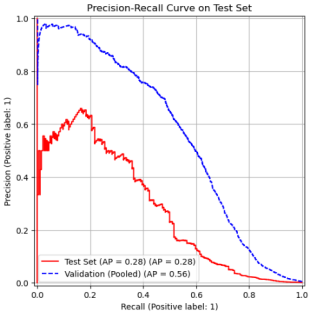
	precision	recall	f1-score	support
0	1.00	1.00	1.00	110960
1	0.34	0.43	0.38	184
accuracy			1.00	111144
macro avg	0.67	0.72	0.69	111144
weighted avg	1.00	1.00	1.00	111144

(b) Threshold = 0.9815

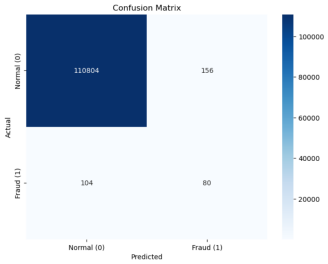
Fig. 16: Classification report of LR model at different thresholds

The performance of the LR model on the test set using default threshold, was even lower than its training results. However, by adopting the optimal threshold identified (0.9815), the classification performance improved significantly. Specifically, the F1-score for the fraud class saw a dramatic increase from 0.04 to 0.38, primarily driven by a substantial rise in Precision (from 0.02 to 0.34). This improvement came at the cost of a typical trade-off, where Recall was reduced by nearly half, dropping from 0.81 to 0.43.

The results show that the LR model is not good enough for real-time, one-at-a-time data processing. The PR-AUC chart comparing validation and test sets clearly shows how the model fails to predict unseen data, the fraud characteristics the model learned during training become less effective at detecting fraud in the test set.



(a) PR-AUC curve



(b) Confusion Matrix

Fig. 17: LR model performance on test set (1)

Moving on to the Random Forest models, we evaluated two distinct configurations. First, we will examine the performance of the more straightforward model - cost-weighted adjustments.

---RESULT ON TEST SET---

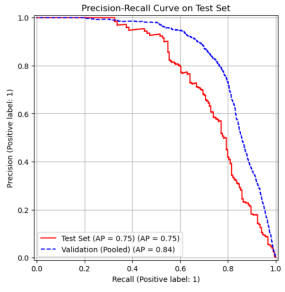
	precision	recall	f1-score	support
0	1.00	1.00	1.00	110960
1	0.42	0.80	0.55	184
accuracy			1.00	111144
macro avg	0.71	0.90	0.77	111144
weighted avg	1.00	1.00	1.00	111144

(a) balanced

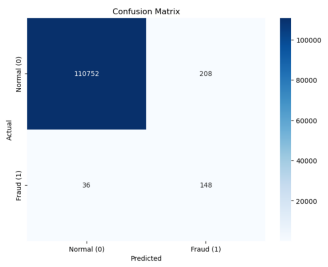
	precision	recall	f1-score	support
0	1.00	1.00	1.00	110960
1	0.55	0.77	0.64	184
accuracy			1.00	111144
macro avg	0.78	0.89	0.82	111144
weighted avg	1.00	1.00	1.00	111144

(b) balanced_subsample

The RF models demonstrate significantly higher efficiency compared to the LR model, confirming my earlier assumption regarding LR's limitations with this specific fraud detection problem. The classification report reveals that to detect nearly 80% (36 missed cases) of fraudulent transactions, the firm only has to sacrifice about half of its precision.



(a) PR-AUC curve



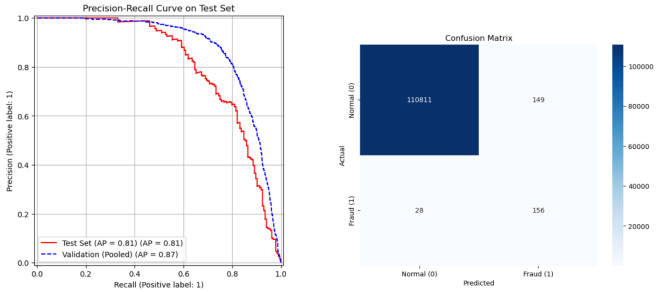
(b) Confusion Matrix

Fig. 19: RF model performance on test set (1)

Moreover, the model's performance on the test set remains highly consistent with the validation results. In the chart, the

test set curve (red line) closely tracks the validation curve (blue line), indicating strong generalization capability and minimal overfitting.

Furthermore, the RF model with hybrid sampling and cost-weighting demonstrates superior performance. Even as we adjust the threshold to capture more fraud, the model effectively sustains high precision, achieving an impressive Average Precision (AP) of 0.81 on unseen data.



(a) PR-AUC curve **(b)** Confusion matrix

Fig. 20: Results of RF model with hybrid sampling at default thresholds

At the default threshold, the model missed only 28 cases, achieving an impressive Recall. However, this high sensitivity led to a lower Precision score of 0.51, resulting in 149 false detections (False Positives).

At the selected optimal threshold, the model successfully identifies 74% of fraudulent transactions. Although this resulted in 47 more missed cases than the default threshold, the precision improved significantly to 0.69 with only 63 false alarms. This strategic trade-off resulted in a more balanced F1-score of 0.71. tion problem, successfully balancing high detection rates with operational efficiency.

	precision	recall	f1-score	support
0	1.00	1.00	1.00	110960
1	0.69	0.74	0.71	184
accuracy			1.00	111144
macro avg	0.84	0.87	0.86	111144
weighted avg	1.00	1.00	1.00	111144

Fig. 21: Classification result

```
rus = RandomUnderSampler(sampling_strategy=0.01, random_state=42)
smote = SMOTE(sampling_strategy=0.1, random_state=42)

rf_pipe_6 = Pipeline([
    ('prep', preprocessor_v2),
    ('undersample', rus),
    ('smote', smote),
    ('rf', RandomForestClassifier(
        n_estimators=100,
        min_samples_leaf = 4,
        max_depth=20,
        class_weight='balanced',
        random_state=42,
        n_jobs=1
    ))
])
```

Fig. 22: Final Random Forest configuration

Due to its robust performance, the Random Forest configuration (Fig. 22)—which integrates SMOTE, Under-sampling, and cost-weighted adjustments - emerged as the most efficient model. It stands as the optimal solution for this credit card fraud detec

6.2. Dataset 2

Once again, applying a custom threshold significantly improves model efficiency. However, a threshold as high as 0.9987 may be impractical for real-world applications. When a detection model only captures transactions with fraud probabilities exceeding 99%, it suggests a potential over-confidence issue, where the model's predicted probabilities do not align with the true likelihood of fraud.

--- Result at default threshold ---

	precision	recall	f1-score	support
0	1.00	0.97	0.99	56672
1	0.04	0.88	0.08	74
accuracy			0.97	56746
macro avg	0.52	0.93	0.53	56746
weighted avg	1.00	0.97	0.99	56746

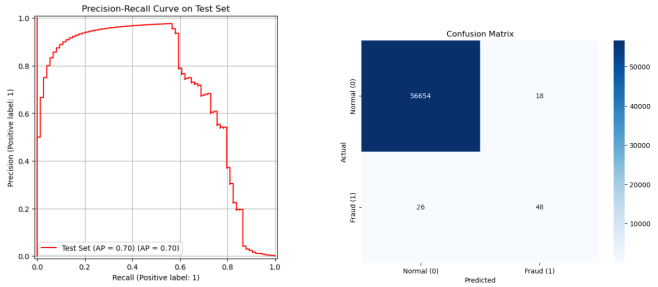
(a) Threshold = 0.5

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56672
1	0.73	0.65	0.69	74
accuracy			1.00	56746
macro avg	0.86	0.82	0.84	56746
weighted avg	1.00	1.00	1.00	56746

(b) Threshold = 0.9987

Fig. 23: Classification report of LR model at different thresholds

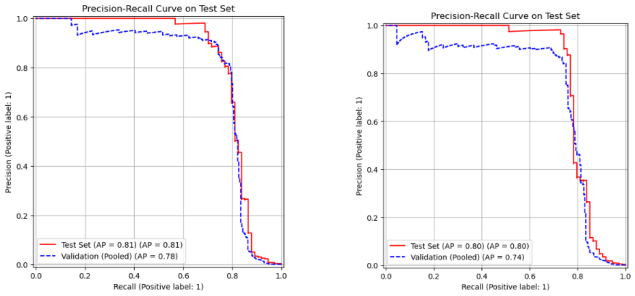
The LR results at the optimized threshold are encouraging, yielding an F1-Score of 0.69 (FN: 18 cases and FP: 26 cases). Analysis of the PR-AUC suggests that the model maintains high precision for approximately 60% of fraudulent transactions; however, it encounters difficulty in classifying the remaining, more complex cases without increasing the false alarm rate.



(a) PR-AUC curve **(b)** Confusion Matrix

Fig. 24: LR model performance on test set (2)

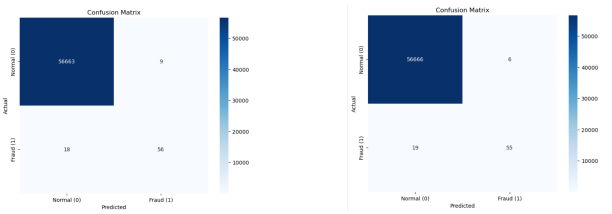
Both Random Forest models yielded competitive results, demonstrating strong generalization across different feature architectures



(a) cost-weight **(b)** hybrid sampling

Fig. 25: RF model performance on test set (2)

The results indicate that with the PCA dataset, which contribute to reduce noise of dataset, sampling method does not bring any significantly improvement.



(a) cost-weight **(b)** hybrid sampling

Fig. 26: RF model performance on test set (2)

Specifically, the cost-weighted RF model achieved a high detection rate by correctly identifying 56 out of 74 fraud cases while maintaining an extremely low false positive rate, with only 9 legitimate transactions misclassified as fraud. This performance confirms that direct cost adjustment is sufficient to handle imbalance in a low-noise, abstracted feature space without the overhead of synthetic sampling

```
rf_pipe_2 = Pipeline([
    ('prep', preprocessor),
    ('rf', RandomForestClassifier(
        n_estimators=100,
        max_depth=10,
        min_samples_leaf = 10,
        class_weight='balanced',
        random_state=42,
        n_jobs=1
    ))
])
```

Fig. 27: RF configuration on Dataset 2

VII. Conclusion and Future Work

This study demonstrates that Random Forest consistently outperforms Logistic Regression in capturing complex fraud patterns across different data architectures. A pivotal finding is the interplay between data processing and sampling: for PCA-anonymized data, the cost-weighted approach proved optimal due to inherent noise reduction, while the simulated raw dataset required hybrid sampling to achieve a competitive F1-Score of 0.71 (compared to 0.81 on PCA). By implementing 3-fold time-series validation instead of random splits, we mitigated data leakage, ensuring a realistic assessment of the model's predictive power.Furthermore, careful threshold tuning was identified as essential, as a custom threshold significantly improved the balance between fraud detection and false alarms

Future research will focus on validating these pipelines on real-world transactional features—such as geolocation and device fingerprints—which are often lost in PCA or simulation. Additionally, integrating deep learning architectures like LSTMs or GNNs could further exploit the temporal and relational dependencies of modern financial transactions. Ultimately, an effective fraud detection system relies on an end-to-end, time-aware pipeline that balances performance with feature interpretability.

To conclusion, an automated fraud detection system's success relies not only on the algorithm itself but on a end-to-end pipeline that respects temporal characteristics. By preventing data leakage and focusing on feature interpretability, this study demonstrates that a well-tuned, time-aware model is significantly more reliable for the dynamic nature of modern financial security.

References

[1] M. Chetan, Jan.2026 "Credit card Fraud data", Kaggle.[Online]. Available at: https://www.kaggle.com/datasets/chetanmittal033/credit-card-fraud-data. [Accessed at: 20th Apr 2026]

[2] Worldline and the Machine Learning Group of ULB ((Universite Libre de Bruxelles). 2016, "Credit Card Fraud Detection" openbigdata.org. [Online]. Available: https://openbigdata.org/resource/credit-card-fraud-detection/ [Accessed: 20th Apr 2026]

[3] Dornadula V.N, Geetha S, "Credit Card Fraud Detection using Machine Learning Algorithms" in 2019 Int. Conf. on recent trends in advanced computing, ICRTAC 2019, doi: 10.1016/j.procs.2020.01.057

[4] A. A. Taha and S. J. Malebary, "An Intelligent Approach to Credit Card Fraud Detection Using an Optimized Light Gradient Boosting Machine," IEEE Access, vol. 8, pp. 42579–42587, 2020, doi: 10.1109/ACCESS.2020.2976810.

[5] Benchaji, I., Douzi, S., El Ouahidi, B. et al. Enhanced credit card fraud detection based on attention mechanism and LSTM deep model. J Big Data 8, 151 (2021). https://doi.org/10.1186/s40537-021-00541-8

[6] Whitrow, C. and Hand, David and Juszczak, P. and Weston, David and Adams, Niall. (2009). Transaction aggregation as a strategy for credit card fraud detection. Data Mining and Knowledge Discovery. 18. 30-55. 10.1007/s10618-008-0116-z.

[7] Dal Pozzolo A, Boracchi G, Caelen O, Alippi C, Bontempi G. Credit Card Fraud Detection: A Realistic Modeling and a Novel Learning Strategy. IEEE Trans Neural Netw Learn Syst. 2018 Aug;29(8):3784-3797. doi: 10.1109/TNNLS.2017.2736643. Epub 2017 Sep 14. PMID: 28920909.

[8] S. Bhattacharyya, S. Jha, K. Tharakunnel, and J. C. Westland, "Data mining for credit card fraud: A comparative study," Decision Support Systems, vol. 50, no. 3, pp. 602–613, 2011. doi: 10.1016/j.dss.2010.08.008.